

CO-3-AT-2

Name:A.Shiva Shankar Reddy

Reg:No:192425174

Course Code: CSA0613

Course Name:DAA

Q1. Case Study: E-Commerce Product Sorting System (Merge Sort)

Aim

To implement the **Merge Sort** algorithm for sorting products based on price and ratings in an e-commerce platform and analyze its efficiency using the Divide and Conquer technique.

Algorithm

1. Read the product details (Name, Price, Rating).
2. Divide the product list into two equal halves.
3. Recursively divide each half until only one element remains.
4. Merge the divided lists by comparing prices.
5. If prices are equal, compare ratings.
6. Display the sorted products.

Python Program

Python

```
class Product:
```

```
    def __init__(self, name, price, rating):
```

```
        self.name = name
```

```
        self.price = price
```

```
        self.rating = rating
```

```
def merge_sort(products):
```

```
    if len(products) > 1:
```

```
        mid = len(products) // 2
```

```
        left = products[:mid]
```

```
        right = products[mid:]
```

```
        merge_sort(left)
```

```
        merge_sort(right)
```

```
i = 0
j = 0
k = 0
while i < len(left) and j < len(right):
    if left[i].price < right[j].price:
        products[k] = left[i]
        i += 1
    elif left[i].price > right[j].price:
        products[k] = right[j]
        j += 1
    else:
        if left[i].rating >= right[j].rating:
            products[k] = left[i]
            i += 1
        else:
            products[k] = right[j]
            j += 1
    k += 1
while i < len(left):
    products[k] = left[i]
    i += 1
    k += 1
while j < len(right):
    products[k] = right[j]
```

```

        j += 1

        k += 1

n = int(input("Enter number of products: "))

products = []

for i in range(n):

    print("\nProduct", i + 1)

    name = input("Name: ")

    price = float(input("Price: "))

    rating = float(input("Rating: "))

    products.append(Product(name, price, rating))

merge_sort(products)

print("\nSorted Product List")

print("-----")

print("Name\tPrice\tRating")

for p in products:

    print(p.name, "\t", p.price, "\t", p.rating)

```

Sample Input

Enter number of products: 5

Product 1

Name: Laptop

Price: 60000

Rating: 4.5

Product 2

Name: Mobile

Price: 25000

Rating: 4.8

Product 3

Name: Watch

Price: 5000

Rating: 4.2

Product 4

Name: Camera

Price: 60000

Rating: 4.8

Product 5

Name: Speaker

Price: 7000

Rating: 4.3

Sample Output

Sorted Product List

Name	Price	Rating
------	-------	--------

Watch	5000.0	4.2
-------	--------	-----

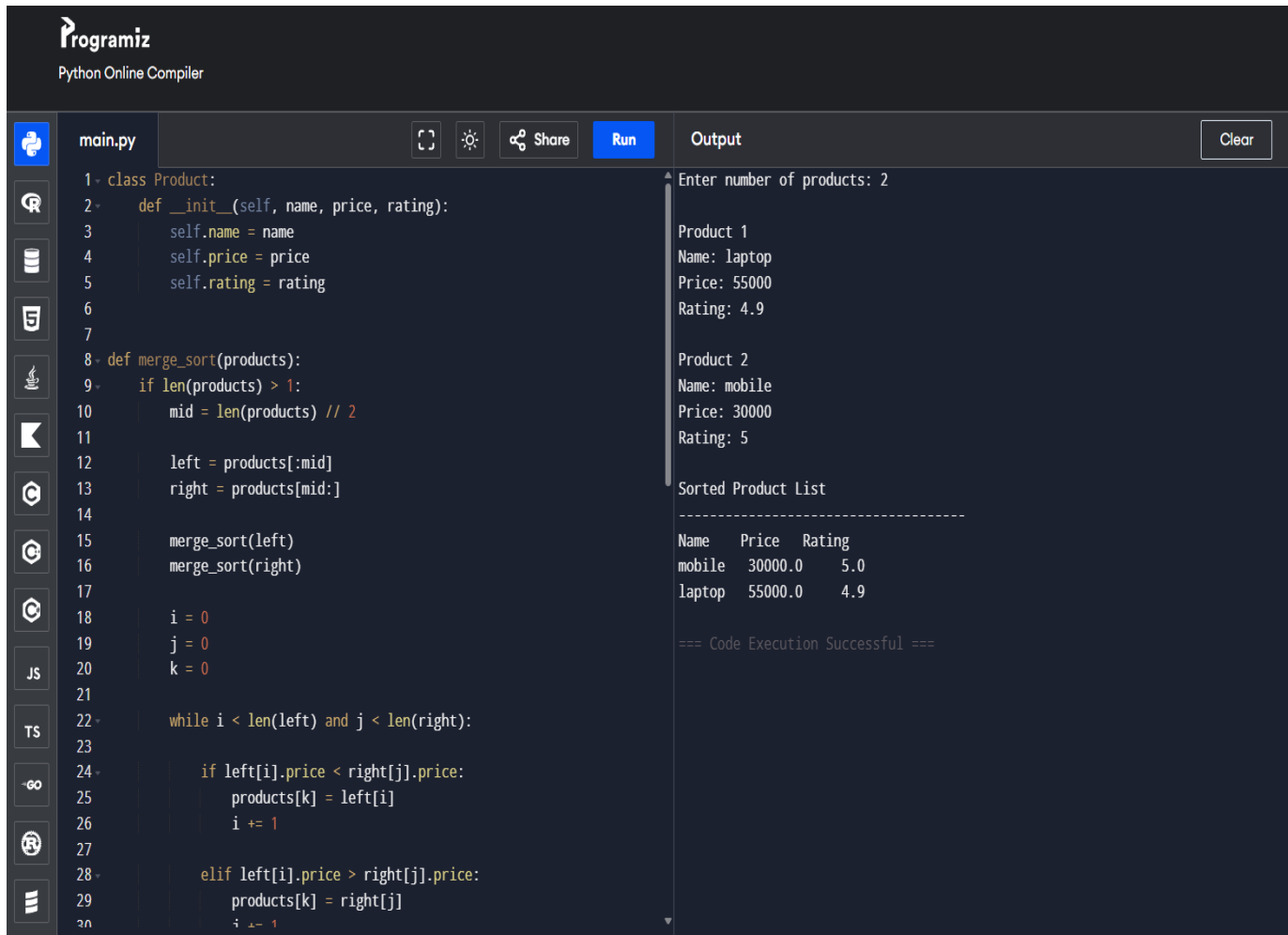
Speaker	7000.0	4.3
---------	--------	-----

Mobile	25000.0	4.8
--------	---------	-----

Camera	60000.0	4.8
--------	---------	-----

Laptop	60000.0	4.5
--------	---------	-----

OUTPUT



The screenshot shows the Programiz Python Online Compiler interface. The code editor on the left contains a Python script for a merge sort algorithm. The output panel on the right shows the execution results, including user input, product details for two items, and a sorted list of products by price.

```
1- class Product:
2-     def __init__(self, name, price, rating):
3-         self.name = name
4-         self.price = price
5-         self.rating = rating
6-
7-
8- def merge_sort(products):
9-     if len(products) > 1:
10-         mid = len(products) // 2
11-
12-         left = products[:mid]
13-         right = products[mid:]
14-
15-         merge_sort(left)
16-         merge_sort(right)
17-
18-         i = 0
19-         j = 0
20-         k = 0
21-
22-         while i < len(left) and j < len(right):
23-
24-             if left[i].price < right[j].price:
25-                 products[k] = left[i]
26-                 i += 1
27-
28-             elif left[i].price > right[j].price:
29-                 products[k] = right[j]
30-                 j += 1
```

Output:

```
Enter number of products: 2

Product 1
Name: laptop
Price: 55000
Rating: 4.9

Product 2
Name: mobile
Price: 30000
Rating: 5

Sorted Product List
-----
Name    Price    Rating
mobile  30000.0    5.0
laptop  55000.0    4.9

=== Code Execution Successful ===
```

Analysis

Problem

An e-commerce website needs to sort millions of products based on price and ratings for faster customer searches.

Key Issues

- Efficient sorting for large datasets.
- Stable sorting when prices are equal.
- High scalability.
- Fast performance.

Divide and Conquer

Merge Sort divides the product list into smaller halves, sorts each half recursively, and merges them into a single sorted list.

Time Complexity

Case	Complexity
Best Case	$O(n \log n)$
Average Case	$O(n \log n)$
Worst Case	$O(n \log n)$

Space Complexity

$O(n)$

Performance

- Suitable for very large datasets.
- Consistent performance.
- Stable sorting preserves the order of products with equal prices.
- Can be parallelized for distributed systems.

Solution Design

1. Store product details.
2. Apply Merge Sort.
3. Compare prices first.
4. If prices are equal, compare ratings.

5. Display the sorted product list.

Justification

Merge Sort is suitable because:

- Stable sorting algorithm.
- Guaranteed **$O(n \log n)$** performance.
- Efficient for large-scale e-commerce databases.
- Easily scalable for distributed environments.
- Produces accurate and consistent sorting results.

Conclusion

Merge Sort efficiently sorts millions of products using the Divide and Conquer approach. It provides stable sorting, predictable **$O(n \log n)$** time complexity, and excellent scalability, making it an ideal choice for e-commerce product sorting systems.